

Periféricos y Dispositivos de Interfaz Humana

Universidad de Granada - Curso 2025/2026

Práctica 5

Experimentación con el sistema de salida de
sonido

Autor: Alejandro López Jiménez

Índice

1. Introducción	2
2. Requisitos mínimos	2
2.1. Ejercicio 1	2
2.2. Ejercicio 2	3
2.3. Ejercicio 3	4
2.4. Ejercicio 4	5
2.5. Ejercicio 5	5
2.6. Ejercicio 6	6
3. Requisitos ampliados	6
3.1. Ejercicio 7	7
3.2. Ejercicio 8	7

1. Introducción

El objetivo de esta práctica es entender la estructura de los archivos de sonido WAV y trabajar con ellos realizando distintas operaciones, como representar gráficamente su forma de onda, filtrar frecuencias, juntar sonidos, añadir el efecto de eco, entre otras.

Para ello, en mi caso, como sugiere el guión, he utilizado el lenguaje R a través del entorno de programación R Studio.

En R Studio se pueden utilizar distintas librerías para añadir funcionalidades al programa. Las utilizadas en esta práctica son:

- **tuneR**: sirve para leer, escribir y manipular archivos de audio. Permite cargar sonidos, acceder a sus muestras y exportar resultados.
- **seewave**: destinada al análisis y procesamiento de señales de audio. Incluye funciones para visualizar formas de onda y aplicar filtros de frecuencia.
- **audio**: sirve para la reproducción y gestión básica de audio desde R, permitiendo escuchar los sonidos cargados o procesados durante el análisis.

Para cumplir con los objetivos de la práctica se proponen una serie de requisitos mínimos y requisitos ampliados, los cuales he incluido al completo en un solo script llamado **practica5.r**.

He establecido el path a la carpeta de trabajo mediante el siguiente comando:

```
setwd("C:/Users/estudio/Documents/PDIH/P5")
```

De esta forma puedo leer archivos desde esa carpeta y exportarlos ahí también.

2. Requisitos mínimos

Los requisitos mínimos para esta práctica se dividen en una serie de ejercicios que son los siguientes:

1. Crear dos ficheros de sonido (WAV) para realizar los siguientes ejercicios. En el primero debe escucharse el nombre de la persona que realiza la práctica. En el segundo debe escucharse el apellido.
2. Leer los dos ficheros de sonido creados y dibujar la forma de onda de ambos sonidos (por separado).
3. Obtener la información de las cabeceras de ambos sonidos.
4. Unir ambos sonidos en uno nuevo para escuchar el nombre y apellido correctamente.
5. Dibujar la forma de onda de la señal y reproducir el sonido resultante (una vez unidos).
6. Almacenar el sonido resultante en un archivo nuevo llamado "basico.wav".

2.1. Ejercicio 1

Para hacer este ejercicio, en el que se pide crear dos archivos de sonido WAV en los que se escuche mi nombre y mis apellidos, mi primera idea fue grabarlos usando mi micrófono y la página web: <https://products.aspose.app/audio/es/voice-recorder/wav>

Sin embargo, como estos dos archivos son necesarios para posteriores ejercicios, era necesario que tuvieran una calidad y una duración aceptables y mediante este método no lo logré. Se escuchaban mal y la representación de sus ondas era mala al tener ruido de fondo y una duración

demasiado larga. Aún así decidí mantenerlos en la carpeta de Sonidos del repositorio con los nombres `nombre_grabado.wav` y `apellidos_grabado.wav`.

Los que finalmente usé, fueron generados por la página recomendada por el profesor: <https://speechgen.io/>

Los nombres de estos archivos son `nombre.wav` y `apellidos.wav`.

2.2. Ejercicio 2

Para leer los archivos de sonido generados en el ejercicio anterior he utilizado la función `readWave` y he almacenado las lecturas en las variables `nombre` y `apellidos`. Para dibujar la forma de sus ondas he combinado las funciones `extractWave` (para extraer las muestras del sonido) y `plot` (para dibujarlas).

Como a la función `extractWave` hay que pasarle como parámetros la muestra inicial y la muestra final para indicarle la duración (en nuestro caso, completa) he utilizado la función `length` para obtener el número de muestras del audio de cada archivo y los he almacenado en dos variables auxiliares, `nombre_duracion` y `apellidos_duracion`.

El fragmento del script que contiene este ejercicio es el siguiente:

```
6 nombre <- readWave("nombre.wav")
7 apellidos <- readWave("apellidos.wav")
8
9 duracion_nombre <- length(nombre@left)
10 duracion_apellidos <- length(apellidos@left)
11
12 plot( extractWave(nombre, from = 1, to = duracion_nombre) )
13 plot( extractWave(apellidos, from = 1, to = duracion_apellidos) )
```

Las representaciones gráficas de las ondas son las siguientes:

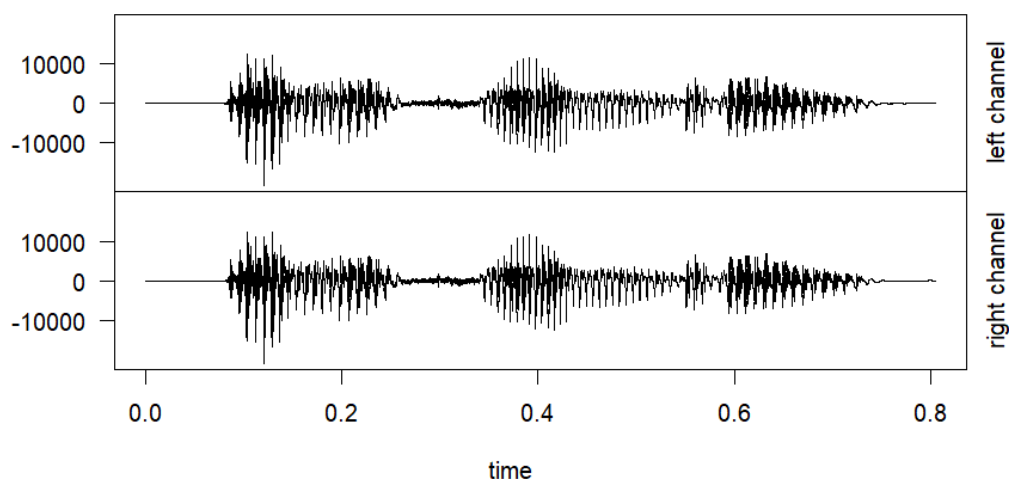


Figura 1: Forma de onda del archivo `nombre.wav`

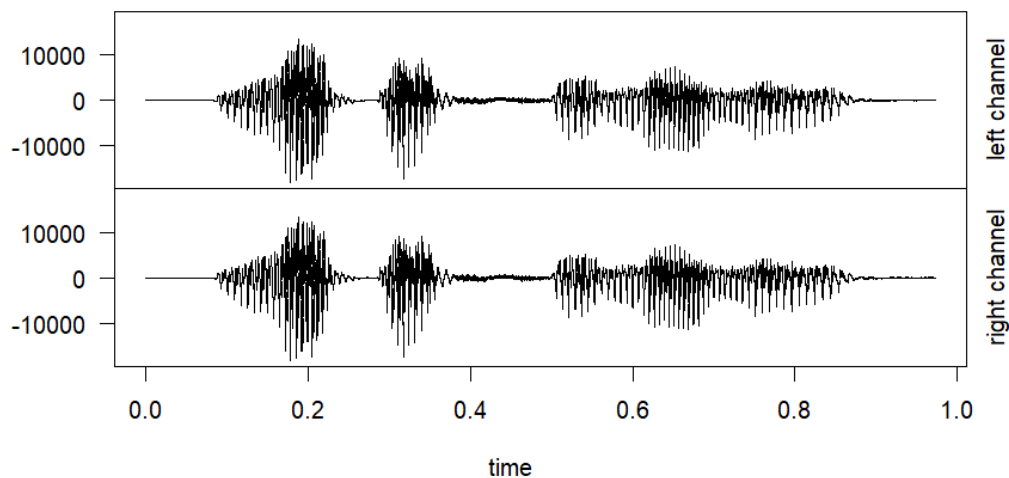


Figura 2: Forma de onda del archivo apellidos.wav

2.3. Ejercicio 3

Este ejercicio es muy simple. Para obtener la información de las cabeceras de los archivos anteriores he aplicado la función `str` sobre las variables que almacenan las lecturas (`nombre` y `apellidos`).

El fragmento del script que contiene este ejercicio es el siguiente:

```
16 str(nombre)
17 str(apellidos)
```

Los resultados son los siguientes:

```
str(nombre)
Formal class 'Wave' [package "tuneR"] with 6 slots
 ..@ left      : int [1:38639] 0 0 0 0 0 0 0 0 0 0 ...
 ..@ right     : int [1:38639] 0 0 0 0 0 0 0 0 0 0 ...
 ..@ stereo    : logi TRUE
 ..@ samp.rate : int 48000
 ..@ bit       : int 16
 ..@ pcm       : logi TRUE

str(apellidos)
Formal class 'Wave' [package "tuneR"] with 6 slots
 ..@ left      : int [1:46703] 0 0 0 0 0 0 0 0 0 0 ...
 ..@ right     : int [1:46703] 0 0 0 0 0 0 0 0 0 0 ...
 ..@ stereo    : logi TRUE
 ..@ samp.rate : int 48000
 ..@ bit       : int 16
 ..@ pcm       : logi TRUE
```

La interpretación de los resultados es la siguiente:

- **left:** Canal izquierdo del audio. Es un vector de enteros (`int`) indicando el número de

muestras (38.639 para el primer archivo y 46.703 para el segundo). Cada número representa la amplitud de onda en un instante.

- **right**: Canal derecho del audio. Lo mismo que para el izquierdo. Como el audio es estéreo existen dos canales.
- **stereo**: es un booleano que indica si el archivo es estéreo o no, como en nuestro caso ambos archivos lo son ambos muestran el valor TRUE.
- **samp.rate**: indica la frecuencia de muestreo, es decir, cuántas muestras se capturan por segundo.
- **bit**: indica el número de bits usado para almacenar cada muestra, en nuestro caso ambos archivos usan 16 bits.
- **pcm**: indica si el audio utiliza codificación PCM (Pulse Code Modulation).

2.4. Ejercicio 4

Para unir ambos sonidos simplemente he utilizado la función `pastew`, pasándole como parámetros las variables que almacenan las lecturas (`nombre` y `apellidos`) e indicando en el output que el formato del archivo resultante sea también Wave (WAV). El resultado lo he almacenado en la variable `nombre_apellidos`.

La línea de código es la siguiente:

```
20 nombre_apellidos <- pastew(apellidos, nombre, output="Wave")
```

2.5. Ejercicio 5

Para hacer este ejercicio he partido de la variable con la unión de los sonidos creada anteriormente (`nombre_apellidos`) y le he aplicado las funciones `extractWave` y `plot` combinadas de la misma manera que en el Ejercicio 2.

Nuevamente he usado `length` para obtener el número de muestras y lo he almacenado en la variable `duracion_nombre_apellidos` para pasársela como parámetro a `extractWave`. Finalmente he usado la función `listen` para reproducir el sonido.

El fragmento del script que contiene este ejercicio es el siguiente:

```
23 duracion_nombre_apellidos <- length(nombre_apellidos@left)
24 plot( extractWave(nombre_apellidos, from = 1, to = duracion_nombre_
      apellidos) )
25
26 listen(nombre_apellidos)
```

La representación gráfica de la onda es la siguiente:

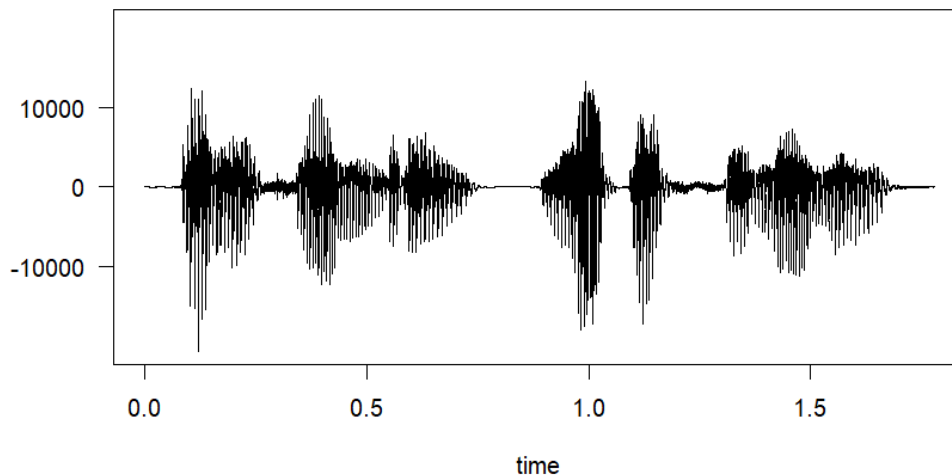


Figura 3: Forma de onda del archivo con nombre y apellidos unidos

2.6. Ejercicio 6

Este ejercicio nuevamente es muy corto y solo ocupa una línea de código. Para almacenar la unión de ambos sonidos en un fichero Wave llamado `basico.wav` simplemente he usado la función `writeWave` pasándole como parámetros la variable que almacena la unión de ambos sonidos (`nombre_apellidos`) y el nombre deseado del archivo (`basico.wav`).

La línea de código es la siguiente:

```
29 writeWave(nombre_apellidos, "basico.wav")
```

3. Requisitos ampliados

Para los requisitos ampliados se proponen dos ejercicios, denominados en esta memoria Ejercicio 7 y Ejercicio 8, que consisten en:

- **Ejercicio 7:** Pasarle un filtro de frecuencia para eliminar las frecuencias entre 10.000Hz y 20.000Hz. Almacenar la señal obtenida como un fichero WAV denominado “filtrado.wav”.

Nota: no se especifica a qué archivo hay que pasarle el filtro de frecuencia por lo que he supuesto que se refería al generado en el ejercicio 6 (`basico.wav`).

- **Ejercicio 8:** Tomar el sonido que se creó antes para aplicarle el efecto de eco. Guardar ese sonido en un archivo nuevo llamado “eco.wav”. A continuación, se le debe dar la vuelta al sonido y almacenarlo como un fichero llamado “alreves.wav”.

3.1. Ejercicio 7

Para aplicar un filtro de frecuencia al archivo `basico.wav` he utilizado la función `bwfilter` y he almacenado el resultado en la variable `filtrado`.

Los parámetros que le he pasado a `bwfilter` son los siguientes:

- `nombre_apellidos`: audio sobre el que se aplica el filtro.
- `f = nombre_apellidos@samp.rate`: frecuencia de muestreo del audio.
- `channel = 1`: canal sobre el que se trabaja. 1 corresponde al canal izquierdo, 2 sería el derecho.
- `n = 1`: orden del filtro Butterworth. Cuanto mayor es este valor, más brusca es la transición entre las frecuencias que se conservan y las que se eliminan.
- `from = 10000`: frecuencia inferior del rango de filtrado (10 kHz).
- `to = 20000`: frecuencia superior del rango de filtrado (20 kHz).
- `bandpass = TRUE`: indica que se conservan únicamente las frecuencias comprendidas entre `from` y `to`.
- `listen = FALSE`: indica que no se reproduzca automáticamente el resultado tras aplicar el filtro.
- `output = "Wave"`: indica que se devuelva como resultado un objeto de tipo Wave.

Tras esto he usado de nuevo la función `writeWave` para almacenar el resultado en el archivo `filtrado.wav`.

El fragmento del script que contiene este ejercicio es el siguiente:

```
32 filtrado <- bwfilter(nombre_apellidos,f=nombre_apellidos@samp.rate,  
    channel=1, n=1, from=10000, to=20000, bandpass=TRUE, listen = FALSE,  
    output = "Wave")  
33 writeWave(filtrado, "filtrado.wav")
```

3.2. Ejercicio 8

Para hacer este ejercicio he usado las funciones `echo` y `revw`.

He partido de la variable creada en el ejercicio anterior que almacena el sonido filtrado (`filtrado`) y se la he pasado a la función `echo` junto con estos otros parámetros:

- `f = filtrado@samp.rate`: frecuencia de muestreo del audio.
- `amp = c(0.8, 0.4, 0.2)`: intensidad de cada eco. 0,8 indica un 80 % respecto al audio original y va disminuyendo a 40 % y finalmente 20 %.
- `delay = c(1, 2, 3)`: retraso de cada eco respecto al sonido original.
- `output = "Wave"`: devuelve el resultado como un objeto Wave.

La aplicación de esta función la he almacenado en la variable `eco` para después usarla en la función `writeWave` para crear el archivo `eco.wav` como pide el ejercicio.

Después esta variable `eco` se la he pasado como parámetro a la función `revw` junto con el parámetro `output="Wave"` (para indicarle que el archivo resultante sea también de tipo Wave) para darle la vuelta al sonido y que se escuche al revés. He almacenado el resultado en la variable `alreves`

y nuevamente he vuelto a usar `writeWave` para almacenar este sonido invertido resultante en el archivo `alreves.wav`.

Sin embargo un problema que noté es que en los archivos generados no se escuchaba nada, lo cual solucioné ajustando la amplitud de ambas señales antes de exportarlas, multiplicando todas las muestras del canal izquierdo de la variable `eco` por 30000 y normalizando la amplitud de la variable `alreves` al rango permitido para ficheros WAV de 16 bits mediante la función `normalize()`, logrando que así sí se oyeran los archivos resultantes. Esto último no lo pedía la práctica, aunque en los ejemplos del seminario se utilizaba, por lo que me pareció adecuado e incluso posiblemente necesario aplicarlo a este ejercicio.

El fragmento del script que contiene este ejercicio es el siguiente:

```
36 eco <- echo(filtrado, f=filtrado@samp.rate, amp=c(0.8,0.4,0.2), delay=c
    (1,2,3), output="Wave")
37 eco@left <- eco@left * 30000
38 writeWave(echo, "eco.wav")
39
40 alreves <- revw(echo, output="Wave")
41 alreves <- normalize(alreves, unit = "16")
42 writeWave(alreves, "alreves.wav")
```

Los archivos resultantes de esta práctica se pueden encontrar en las carpetas Sonidos e Imágenes del repositorio en Github.